Now let's get into a practical example. Consider the code given below:

```
import gtk
wnd = gtk.Window()
vbox = gtk.VBox()
lbl1 = gtk.Label("This is label 1")
lbl2 = gtk.Label("This is label 2")
vbox.add(lbl1)
vbox.add(lbl2)
wnd.add(vbox)
wnd.set_title("Adding Multiple Widgets")
wnd.set_size_request(400, 200)
wnd.show_all()
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

Here, we have made use of VBox. That's why Label 2 appeared below Label 1. Try HBox instead of VBox and you will get Label 1 and Label 2 placed on both sides of the window.

Now you have got a basic idea of how to handle multiple widgets. Perform experiments with multiple containers at a time and you will find the process interesting.

Handling events

We've already said that handling events is the major part of GUI programming. Although we didn't try to write any event handlers so far, we've already made a connection—connecting the window's 'destroy' event with the function *gtk.main_quit*. Now, let's write a custom event handler. Our aim is to present a button, which when pressed will change the text inside a label box. The code is given below:

```
import gtk
wnd = gtk.Window()
vbox = gtk.VBox()
btn = gtk.Button("Press Me")
lbl = gtk.Label("You have not pressed the button yet.")
def on_btn_click(widget):
     lbl.set_text("Thank you for pressing the button.");
vbox.add(lbl)
vbox.add(btn)
wnd.add(vbox)
wnd.set_title("Handling Events")
wnd.set_size_request(400, 100)
wnd.show_all()
btn.connect("clicked", on_btn_click)
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

Key points are the function *on_btn_click* and the line used to connect the *clicked* events of the button. Here, the parameter *widget* of the function *on_btn_click* is filled with a reference to the *widget*, which the function got a signal from. In this example, the variable widget acts like the variable *btn*, when the function is called. Consider the re-written event handler:

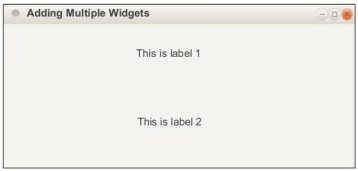

Figure 4: Multiple widgets placed on a single window

```
def on_btn_click(widget):
     lbl.set_text(widget.get_label());
```

widget.get_label() means *btn.get_label()*, since the event handler is invoked by *btn*.

This feature is useful while connecting multiple widgets to a single event handler. Consider the code below:

```
import gtk
wnd = gtk.Window()
vbox = gtk.VBox()
btn1 = gtk.Button("Button 1")
btn2 = gtk.Button("Button 2")
lbl = gtk.Label("You have not pressed any button yet.")
def on_btn_click(widget):
     name = widget.get_name()
     if(name == "button1"):
       lbl.set_text("Thank you for pressing button 1.");
     else:
       lbl.set_text("Thank you for pressing button 2.");
vbox.add(lbl)
vbox.add(btn1)
vbox.add(btn2)
wnd.add(vbox)
wnd.set_title("Handling Events")
wnd.set_size_request(400, 100)
wnd.show_all()
btn1.set_name("button1")
btn2.set_name("button2")
btn1.connect("clicked", on_btn_click)
btn2.connect("clicked", on_btn_click)
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

In the above example, both *btn1* and *btn2* are connected to *on_btn_click*. Even so, we are able to distinguish the calling widget since the variable *widget* will point to either *btn1* or *btn2*, according to the situation. But this alone is not sufficient to identify the caller. That's why we put tags on those buttons, using the function *set_name*.